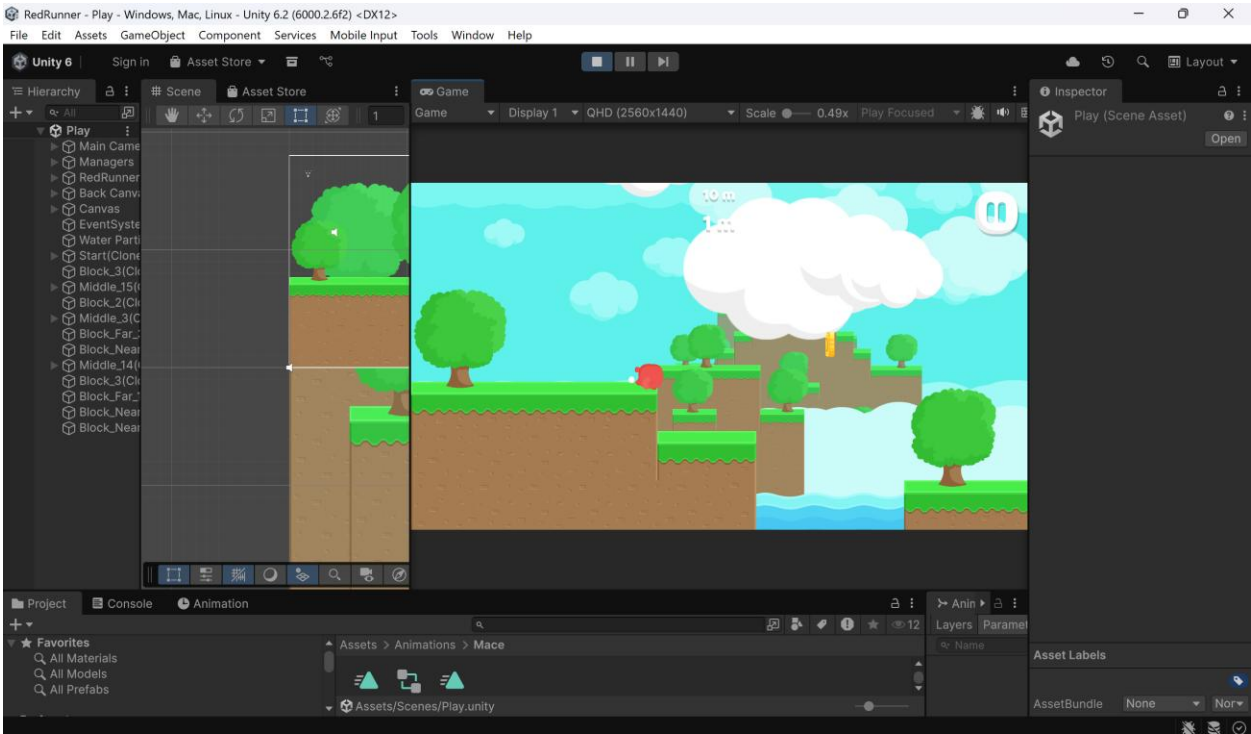


Lab 01: Khám phá dự án Game 2D thực tế với Unity



PHẦN 2: Khám phá cấu trúc dự án

Task 2.1: Cấu trúc thư mục

Câu 1: Liệt kê tất cả các thư mục con trực tiếp trong Assets/ . Mô tả ngắn gọn vai trò của từng thư mục.

Thư mục	Vai trò / Mô tả ngắn gọn
Animations	Lưu trữ các Animation Clips (đoạn hoạt ảnh) và Animation Controllers để điều khiển hoạt động của nhân vật / vật thể
Editor	Chứa các đoạn mã (script) dùng để tùy biến hoặc mở rộng tính năng của chính phần mềm Unity Editor. Các file ở đây sẽ không

	được đưa vào bản build cuối cùng của game.
Fonts	Lưu trữ các tệp phông chữ (như .ttf, .otf) để sử dụng cho giao diện người dùng (UI) hoặc văn bản trong game.
Materials	Chứa các Material (vật liệu), định nghĩa cách bề mặt của vật thể phản ứng với ánh sáng, màu sắc và texture.
Physics Materials 2D	Lưu trữ các vật liệu vật lý 2D, dùng để thiết lập độ ma sát (friction) và độ nảy (bounciness) cho các va chạm trong không gian 2D.
Prefabs	Chứa các Prefab – là các đối tượng game đã được thiết lập sẵn (mẫu), có thể tái sử dụng nhiều lần trong các cảnh khác nhau.
Resources	Thư mục đặc biệt dùng để chứa các tài nguyên có thể được tải lên (load) linh hoạt thông qua mã script bằng lệnh Resources.Load trong khi game đang chạy.
SaveGameFree	Thường là thư mục của một plugin bên thứ ba hoặc hệ thống tự tạo để quản lý việc lưu trữ và tải dữ liệu trò chơi (Save/Load game).

Scenes	Chứa các file cảnh quay của trò chơi (tương đương với các màn chơi hoặc các cấp độ level).
Scripts	Nơi lưu trữ toàn bộ các tệp mã nguồn C# điều khiển logic và tính năng của trò chơi.
Shaders	Chứa các chương trình Shader (mã đồ họa) dùng để tạo ra các hiệu ứng hình ảnh đặc biệt trên bề mặt vật thể.
Sounds	Lưu trữ các tệp âm thanh như nhạc nền (BGM) và hiệu ứng âm thanh (SFX).
Sprites	Chứa các hình ảnh 2D (Textures) đã được định dạng là Sprite để sử dụng cho nhân vật, nền và các yếu tố hình ảnh khác trong game 2D.
Standard Assets	Chứa các gói tài nguyên mẫu do Unity cung cấp sẵn (như các bộ điều khiển nhân vật, hiệu ứng camera cơ bản).

Câu 2: Dự án có bao nhiêu Scene? Liệt kê tên và mô tả vai trò của từng Scene.

Dự án này có 2 Scene:

1. Scene: Play

Mô tả: Scene này hiển thị giao diện người dùng (UI) với tiêu đề game "**RED RUNNER**", nút **Play** lớn ở giữa và các nút chức năng khác như cài đặt, chia sẻ mạng xã hội.

Vai trò: Màn hình Menu chính (Main Menu) của trò chơi.

2. Scene: Creation

Mô tả: Đây là nơi diễn ra các hoạt động của trò chơi. Nó bao gồm môi trường 2D với các nền tảng (platforms), bẫy gai, đồng xu và nhân vật chính là RedRunner

Vai trò: Màn chơi chính (Gameplay Scene).

Câu 3: Thư mục Assets/Scripts/ được tổ chức như thế nào? Vẽ sơ đồ cây thư mục (tree) của thư mục Scripts/

Cách tổ chức thư mục

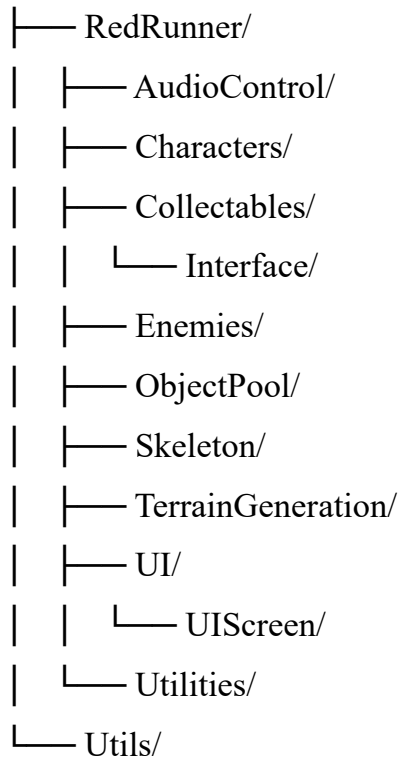
1. Phân loại theo tính năng (**Functional Grouping**): Các mã nguồn không để chung một chỗ mà được chia vào các thư mục con dựa trên nhiệm vụ cụ thể mà chúng đảm nhận trong game (như điều khiển nhân vật, âm thanh, giao diện...).

2. **Cấu trúc phân cấp rõ ràng**: Có một thư mục chính là RedRunner để bao bọc các thành phần logic cốt lõi, giúp tách biệt với các thư mục tiện ích dùng chung như Utils.

3. **Hỗ trợ mở rộng (Scalability)**: Cách chia nhỏ như Collectables/Interface hay UI/UIScreen cho thấy dự án được thiết kế để dễ dàng thêm các vật phẩm hoặc giao diện mới mà không làm rối cấu trúc cũ.

Sơ đồ cây thư mục

Scripts/



Câu 4: Mở thư mục Assets/Prefabs/ . Liệt kê các thư mục con và cho biết mỗi thư mục chứa loại prefab nào.

Thư mục	Loại Prefab chứa bên trong
Background Blocks	Chứa các khối trang trí nền như núi, đồi hoặc các mảng kiến trúc xa xăm để tạo chiều sâu cho game.
Blocks	Chứa các khối kiến trúc tĩnh, có thể là các chướng ngại vật hoặc các phần cấu trúc cố định trong màn chơi.

Collectables	Chứa các vật phẩm mà nhân vật có thể thu thập được, ví dụ như đồng xu (coins) hoặc kho báu.
Enemies	Chứa các đối tượng gây hại cho người chơi, bao gồm kẻ thù di động hoặc các bẫy tĩnh như bẫy gai (spikes).
Grounds	Chứa các mảnh địa hình (nền đất) mà nhân vật chính đi lên. Trong này có thư mục con Dirt With Physics chứa các mảnh đất đã thiết lập sẵn các thành phần vật lý để xử lý va chạm.
Particles	Chứa các hiệu ứng kỹ xảo hình ảnh (hệ thống hạt) như bụi khi chạy, hiệu ứng khi ăn xu hoặc hiệu ứng nổ.

Câu 5: Giải thích khái niệm Prefab trong Unity. Tại sao cần dùng Prefab thay vì tạo GameObject trực tiếp trong Scene?

Trong Unity, **Prefab** là một loại tài nguyên đặc biệt cho phép bạn lưu trữ một **GameObject** hoàn chỉnh cùng với tất cả các thành phần (components), thuộc tính và các đối tượng con của nó như một "bản thiết kế" (blueprint) có thể tái sử dụng.

Lý do tại sao việc sử dụng Prefab lại quan trọng và hiệu quả hơn so với việc tạo GameObject trực tiếp trong Scene:

1. Khả năng tái sử dụng (Reusability)

Thay vì phải thiết lập lại từ đầu các thành phần như *Rigidbody2D*, *BoxCollider2D* hay các đoạn mã điều khiển cho mỗi đối tượng (ví dụ: hàng chục đồng xu trong màn hình Creation), bạn chỉ cần kéo một Prefab từ thư mục Collectables vào Scene.

2. Chỉnh sửa đồng loạt (Mass Editing)

Đây là ưu điểm lớn nhất. Khi bạn thay đổi một thuộc tính trên file Prefab gốc (ví dụ: thay đổi màu sắc của tất cả các khối trong thư mục Background Blocks), tất cả các bản sao (instances) của Prefab đó đang hiện diện trong mọi Scene của dự án sẽ tự động cập nhật theo. Nếu tạo GameObject trực tiếp, bạn sẽ phải sửa thủ công từng đối tượng một.

3. Quản lý dự án sạch sẽ

Như bạn thấy trong cấu trúc thư mục của dự án RedRunner, các đối tượng được chia nhỏ vào các thư mục con như Enemies, Grounds, Particles. Điều này giúp:

- Dễ dàng tìm kiếm và quản lý tài nguyên.
- Tránh việc cửa sổ **Hierarchy** bị quá tải bởi hàng trăm đối tượng rời rạc.

4. Tạo đối tượng linh hoạt (Instantiating)

Prefab là yếu tố bắt buộc nếu bạn muốn tạo ra các đối tượng trong khi game đang chạy (Runtime). Ví dụ: Khi nhân vật RedRunner di chuyển, các mảnh địa hình từ thư mục TerrainGeneration có thể được sinh ra liên tục để tạo cảm giác chạy vô tận.

Task 2.2: Scene và Hierarchy

Câu 6: Liệt kê tất cả các GameObject gốc (root) trong tab Hierarchy.

Main Camera: Đối tượng camera chính quyết định những gì người chơi nhìn thấy trên màn hình.

Managers: Chứa các script quản lý logic toàn cục (thường là Game Manager, Sound Manager, v.v.).

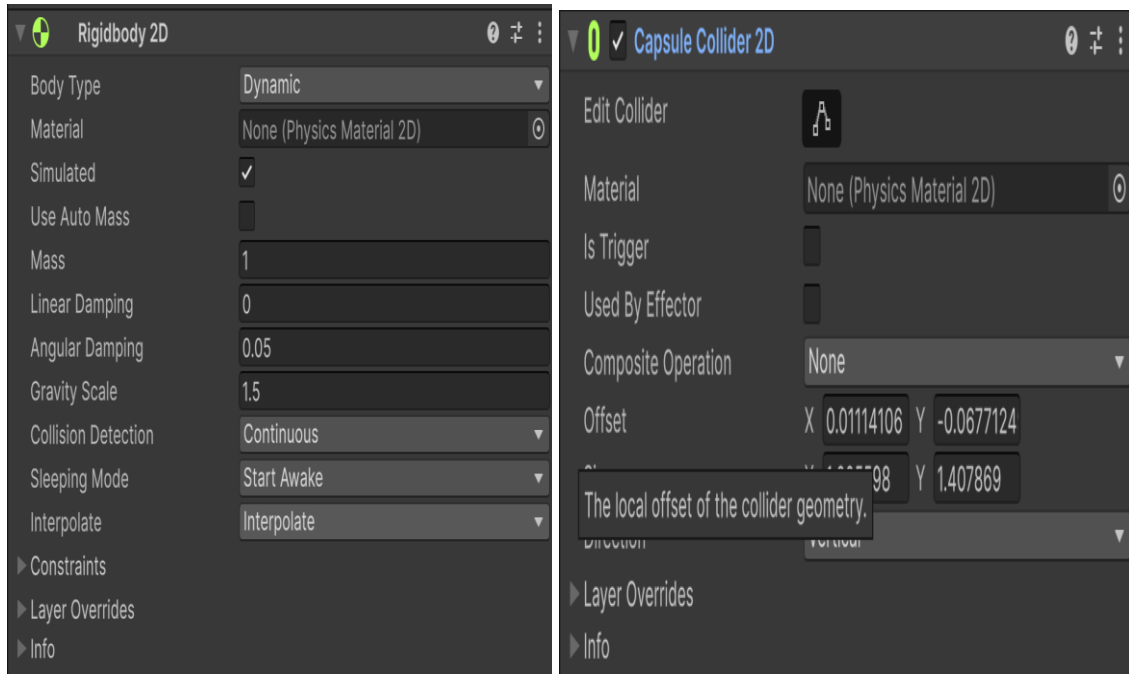
RedRunner: Một bản sao (instance) của nhân vật chính trong Scene này.

Back Canvas: Hệ thống lớp giao diện nằm phía sau.

Canvas: Hệ thống lớp giao diện chính chứa các nút bấm và văn bản (UI).

EventSystem: Thành phần bắt buộc để xử lý các tương tác của người dùng với UI (như nhấn nút Play).

Câu 7: Click chọn nhân vật RedRunner trong Hierarchy → xem tab Inspector. Liệt kê tất cả các Component được gắn trên nhân vật.



Câu 8: Giải thích vai trò của 2 component sau trên nhân vật RedRunner:

1. Rigidbody 2D

Vai trò: Đây là thành phần cho phép nhân vật chịu sự tác động của **vật lý**.

Chức năng cụ thể:

Trọng lực (Gravity): Giúp RedRunner rơi xuống khi nhảy lên hoặc khi đi ra khỏi rìa các khối đất.

Lực và Vận tốc: Cho phép lập trình viên tác động lực để nhân vật chạy hoặc nhảy thông qua code C# trong thư mục Scripts/RedRunner/Characters.

Tương tác động: Giúp nhân vật có khối lượng và phản ứng với các tác động vật lý khác trong Scene Creation.

2. Box Collider 2D

Vai trò: Đây là thành phần xác định **hình dạng vật lý** (vùng va chạm) của nhân vật.

Chức năng cụ thể:

Ngăn xuyên thấu: Giúp RedRunner đứng vững trên các Prefabs thuộc thư mục Grounds mà không bị rơi xuyên qua mặt đất.

Phát hiện va chạm: Kích hoạt các sự kiện khi nhân vật chạm vào bẫy gai (Enemies) hoặc ăn đồng xu (Collectables).

Tạo ranh giới: Nó tạo ra một "hộp" vô hình bao quanh nhân vật để Unity biết đâu là giới hạn cơ thể của RedRunner khi tính toán va chạm.

Câu 9: Tìm các Enemy trong scene (Hierarchy). Liệt kê tên và loại của chúng. Click vào một Enemy bất kỳ → Inspector → ghi nhận các Component được gắn.

Tên chương ngại vật: Saw

Loại: Chương ngại vật tĩnh (Static Hazard). Chúng được thiết kế để gây sát thương hoặc khiến nhân vật "Die" ngay lập tức khi va chạm.

Vị trí: Bạn có thể thấy chúng xuất hiện trên bề mặt các khối đất trong màn chơi

Tên chương ngại vật: Spike Up

Loại: Chương ngại vật tĩnh (Static Hazard). Chúng được thiết kế để gây sát thương hoặc khiến nhân vật "Die" ngay lập tức khi va chạm.

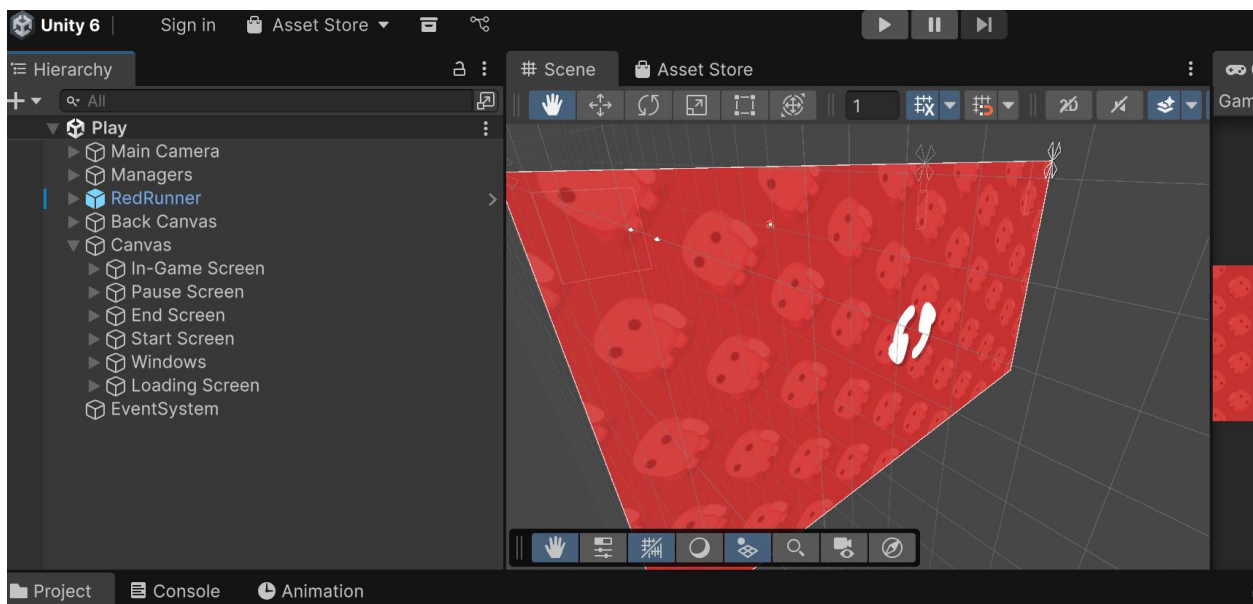
Vị trí: Bạn có thể thấy chúng xuất hiện trên bề mặt các khối đất trong màn chơi

Tên chương ngại vật: Mace

Loại: Chương ngại vật động (Dynamic Hazard). Chúng được thiết kế để rơi xuống khi nhân vật đến gần hoặc khiến nhân vật chết ngay lập tức khi va chạm.

Vị trí: Bạn có thể thấy chúng xuất hiện cách bề mặt các khối đất 1 khoảng nhất định trong màn chơi

Câu 10: Tìm Canvas trong Hierarchy. Nó chứa những UI element nào? Mô tả chức năng của từng element.



1. In-Game Screen

- **Nhiệm vụ:** Hiển thị các thông tin thời gian thực khi người chơi đang thực hiện màn chơi.
- **Các đối tượng con:**
 - **Coin Panel:** Hiển thị số lượng đồng xu đã thu thập được.
 - **Pause Button:** Nút bấm để tạm dừng trò chơi.
 - **High Score Text & Score Text:** Hiển thị điểm số cao nhất và điểm số hiện tại của lượt chơi.

2. Pause Screen

- **Nhiệm vụ:** Hiển thị giao diện tạm nghỉ khi người chơi nhấn nút Pause.
- **Các đối tượng con:**
 - **Backdrop:** Lớp nền mờ phủ lên màn hình chơi.
 - **Sound Button:** Bật/tắt âm thanh.
 - **Home Button & Exit Button:** Quay lại menu chính hoặc thoát trò chơi.
 - **Resume Button:** Tiếp tục lượt chơi đang dang dở.

3. End Screen

- **Nhiệm vụ:** Hiển thị khi nhân vật tử vong hoặc kết thúc lượt chơi.
- **Các đối tượng con:** Chứa các nút chức năng tương tự màn hình tạm dừng như **Home Button**, **Exit Button** và đặc biệt là **Reset Button** để chơi lại ngay lập tức.

4. Start Screen

- **Nhiệm vụ:** Đây là màn hình menu chính xuất hiện khi vừa mở game.
- **Các đối tượng con:**
 - **Logo Image:** Hiển thị logo "RED RUNNER".
 - **Play Button:** Nút chính để bắt đầu vào màn chơi Creation.
 - **Hệ thống nút Share:** Bao gồm Share Google Plus, Twitter, Facebook để chia sẻ trò chơi.
 - **Info & Help Button:** Cung cấp thông tin hướng dẫn và tác giả.

5. Loading Screen

- **Nhiệm vụ:** Xuất hiện trong quá trình chuyển đổi giữa các Scene (ví dụ từ Play sang Creation) để che đi việc tải dữ liệu hệ thống.

Câu 11: GameManager sử dụng design pattern nào? Tìm đoạn code chứng minh.

GameManager sử dụng **Singleton Pattern**

```
#endregion

0 references | Unity Message
void Awake()
{
    if (m_Singleton != null)
    {
        Destroy(gameObject);
        return;
    }
    SaveGame.Serializer = new SaveGameBinarySerializer();
    m_Singleton = this;
    m_Score = 0f;

    if (SaveGame.Exists("coin"))
    {
        m_Coin.Value = SaveGame.Load<int>("coin");
    }
    else
    {

```

Câu 12: Liệt kê các thuộc tính (property) quan trọng mà GameManager quản lý.

- Dữ liệu người chơi:

- m_Coin: Quản lý số lượng tiền vàng thu thập được (sử dụng custom Property<int>).
- m_Score: Điểm số hiện tại của người chơi (dựa trên khoảng cách di chuyển theo trục X).
- m_HighScore: Điểm số cao nhất mọi thời đại.
- m_LastScore: Điểm số của lượt chơi vừa kết thúc.

- Trạng thái hệ thống:

- m_GameStarted: Kiểm tra xem trò chơi đã thực sự bắt đầu chưa.
- m_GameRunning: Kiểm tra xem game đang chạy hay đang bị tạm dừng (Pause).

- `m_AudioEnabled`: Trạng thái bật/tắt âm thanh toàn cục.

- Tham chiếu đối tượng:

- `m_MainCharacter`: Tham chiếu đến nhân vật chính (Character).

Câu 13: Đọc phương thức `StartGame()` . Mô tả bằng lời các bước mà phương thức này thực hiện.

- **Thiết lập trạng thái bắt đầu:** Gán biến `m_GameStarted = true`, xác nhận rằng một lượt chơi mới đã chính thức được kích hoạt.

- **Kích hoạt luồng thời gian:** Gọi hàm `ResumeGame()`.

- **Bật chạy Logic:** Trong hàm `ResumeGame()`, biến `m_GameRunning` được đặt thành `true` và đặc biệt là đặt `Time.timeScale = 1f` để mọi hoạt động vật lý, hoạt ảnh và thời gian trong game quay trở lại tốc độ bình thường.

Câu 14: Khi người chơi thoát game, những dữ liệu nào được lưu lại? Tìm đoạn code liên quan.

- Khi người chơi thoát game (thông qua sự kiện `OnApplicationQuit`), các dữ liệu quan trọng liên quan đến tiến trình của người chơi sẽ được lưu lại bằng thư viện `SaveGameFree`

- + Số lượng Coin hiện có.
- + Điểm số cuối cùng (Last Score) của lượt chơi.
- + Điểm số cao nhất (High Score) đạt được.

```

0 references | 📦 Unity Message
void OnApplicationQuit()
{
    if (m_Score > m_HighScore)
    {
        m_HighScore = m_Score;
    }
    SaveGame.Save<int>("coin", m_Coin.Value);
    SaveGame.Save<float>("lastScore", m_Score);
    SaveGame.Save<float>("highScore", m_HighScore);
}

```

Câu 15: GameManager sử dụng những event nào? Liệt kê tên và mô tả mục đích.

Tên Event	Kiểu Delegate (Handler)	Mục đích
OnReset	ResetHandler	Kích hoạt khi trò chơi được đặt lại (Reset). Nó thông báo cho các hệ thống khác (như TerrainGenerator) biết để dọn dẹp và chuẩn bị cho lượt chơi mới.
OnScoreChanged	ScoreHandler	Thông báo khi điểm số của người chơi thay đổi. Nó truyền đi 3 giá trị: điểm hiện tại (m_Score), điểm cao nhất (m_HighScore), và điểm của

		lượt trước (m_LastScore) để UI cập nhật thanh điểm.
OnAudioEnabled	AudioEnabledHandler	Phát đi thông báo khi trạng thái âm thanh bị thay đổi (Bật hoặc Tắt). Các đối tượng phát nhạc hoặc hiệu ứng âm thanh sẽ lắng nghe event này để tự điều chỉnh âm lượng.

Dựa trên mã nguồn RedCharacter.cs bạn cung cấp, đây là lời giải chi tiết cho các câu hỏi về cơ chế hoạt động của nhân vật:

Câu 16: Cách nhân vật di chuyển ngang

Nhân vật di chuyển bằng cách thay đổi trực tiếp vận tốc (linearVelocity) của thành phần **Rigidbody2D** dựa trên dữ liệu nhập từ bàn phím hoặc joystick.

```

2 references
public override void Move ( float horizontalAxis )
{
    if ( !IsDead.Value )
    {
        float speed = m_CurrentRunSpeed;
        // if ( CrossPlatformInputManager.GetButton ( "Walk" ) )
        // {
        //     speed = m_WalkSpeed;
        // }
        Vector2 velocity = m_Rigidbody2D.linearVelocity;
        velocity.x = speed * horizontalAxis;
        m_Rigidbody2D.linearVelocity = velocity;
        if ( horizontalAxis > 0f )
        {
            Vector3 scale = transform.localScale;
            scale.x = Mathf.Sign ( horizontalAxis );
            transform.localScale = scale;
        }
        else if ( horizontalAxis < 0f )
        {
            Vector3 scale = transform.localScale;
            scale.x = Mathf.Sign ( horizontalAxis );
            transform.localScale = scale;
        }
    }
}

```

Giải thích: * Hàm lấy giá trị horizontalAxis (từ -1 đến 1).

- Nó giữ nguyên vận tốc trục Y hiện tại và chỉ ghi đè vận tốc trục X bằng tích của speed và hướng di chuyển.
- Mathf.Sign giúp lật localScale để nhân vật nhìn sang trái hoặc phải tương ứng với hướng nhấn phím.

Câu 17: Xử lý Nhảy (Jump) và điều kiện


```

2 references
public override void Jump ()
{
    if ( !IsDead.Value )
    {
        if ( m_GroundCheck.IsGrounded )
        {
            Vector2 velocity = m_Rigidbody2D.linearVelocity;
            velocity.y = m_JumpStrength;
            m_Rigidbody2D.linearVelocity = velocity;
            m_Animator.ResetTrigger ( "Jump" );
            m_Animator.SetTrigger ( "Jump" );
            m_JumpParticleSystem.Play ();
            AudioManager.Singleton.PlayJumpSound ( m_JumpAndGroundedAudioSource );
        }
    }
}

```

Điều kiện để nhảy: 1. Nhân vật phải còn sống (!IsDead.Value). 2. Nhân vật phải đang đứng trên mặt đất (m_GroundCheck.IsGrounded). Điều này ngăn việc người chơi nhảy vô hạn giữa không trung (Double Jump).

Câu 18: Vai trò của Mathf.SmoothDamp()

Mục đích: Mathf.SmoothDamp() được sử dụng để thay đổi dần dần vận tốc hiện tại tới vận tốc tối đa một cách mượt mà theo thời gian, thay vì thay đổi đột ngột.

Tại sao không đặt vận tốc trực tiếp?

- Nếu đặt trực tiếp, nhân vật sẽ đạt tốc độ tối đa ngay lập tức, tạo cảm giác di chuyển rất "gắt" và thiếu tự nhiên (giống robot).
- Sử dụng SmoothDamp tạo ra **gia tốc**, giúp nhân vật có độ trễ khi bắt đầu chạy và đạt vận tốc cao dần, mang lại cảm giác vật lý mềm mại và chuyên nghiệp hơn cho game Platformer.

Câu 19: Các sự kiện xảy ra khi nhân vật chết (Die())

Khi phương thức Die() được gọi, một chuỗi các sự kiện sau sẽ xảy ra:

1. **Thay đổi trạng thái:** Biến IsDead.Value được đặt thành true.

2. **Kích hoạt Ragdoll (Xương):** Gọi `m_Skeleton.SetActive(true, ...)`. Lúc này, hoạt ảnh (Animator) bị tắt, Collider chính bị tắt và nhân vật biến thành một hệ thống xương chịu tác động của vật lý (rơi tự do hoặc văng ra).
3. **Hiệu ứng máu:** Nếu tham số `blood` là `true`, một hệ thống hạt (Particle System) chứa máu sẽ được tạo ra tại vị trí nhân vật.
4. **Điều chỉnh Camera:** `CameraController.fastMove` được bật để camera tập trung nhanh vào vị trí nhân vật chết.
5. **Hiệu ứng hình ảnh:** Script sẽ chạy `Coroutine CloseEye()` để thực hiện hiệu ứng nhân vật từ từ nhắm mắt lại.

Câu 20: Có bao nhiêu loại Enemy? Liệt kê tên file và tên class tương ứng.

Tên File	Tên Class tương ứng	Mô tả ngắn
Spike.cs	Spike	Chướng ngại vật gai nhọn.
Saw.cs	Saw	Luối cưa xoay tròn.
Mace.cs	Mace	Quả cầu gai di chuyển (thường là từ trên xuống).
Eye.cs	Eye	Mắt theo dõi nhân vật (đây là một loại enemy đặc biệt không trực tiếp giết bằng va chạm vật lý thông thường).

Câu 21: Lớp Enemy là abstract hay concrete? Giải thích lý do thiết kế này.

Lớp Enemy là một lớp **Abstract** (trừu tượng).

Lý do thiết kế:

- **Tính đa hình:** Mỗi loại kẻ thù (Gai, Cưa, Quả cầu) có cách hiển thị và hành vi khác nhau nhưng đều có chung mục đích là gây hại cho nhân vật. Thiết kế trừu tượng cho phép định nghĩa các phương thức chung như Kill() mà không cần biết cụ thể đối tượng đó là gì.
- **Bắt buộc triển khai:** Bằng cách để public abstract void Kill (Character target);, Unity bắt buộc tất cả các lớp con (Spike, Saw...) phải tự viết logic giết nhân vật riêng của chúng (ví dụ: Spike thì làm nhân vật dính vào gai, Saw thì chỉ đơn thuần là chết).

Câu 22: Enemy phát hiện va chạm với nhân vật bằng cơ chế nào của Unity? Tìm tên phương thức trong code.

Enemy phát hiện va chạm với nhân vật bằng các **Callback vật lý 2D** của Unity. Hệ thống vật lý sẽ tự động gọi các phương thức này khi Collider2D của Enemy tiếp xúc với Collider2D của nhân vật.

Các phương thức cụ thể trong code:

- OnCollisionEnter2D: Gọi khi bắt đầu chạm (tìm thấy trong Saw.cs, Mace.cs).
- OnCollisionStay2D: Gọi liên tục khi vẫn còn đang chạm (tìm thấy trong Spike.cs).

Câu 23: Khi va chạm xảy ra, Enemy gọi phương thức nào trên nhân vật? Viết lại dòng code đó.

Khi va chạm xảy ra và đủ điều kiện để tiêu diệt, Enemy sẽ gọi phương thức Die() trên đối tượng nhân vật (target hoặc character).

```
target.Die (true); // Gọi trong Spike.cs (true nghĩa là có hiệu ứng máu)
// Hoặc
character.Die (false); // Gọi trong Saw.cs
```

Câu 24: So sánh cách xử lý va chạm của Spike và Water. Có gì giống và khác nhau?

Giống nhau:

- Cả hai đều dẫn đến kết quả cuối cùng là gọi hàm Die() để kết thúc lượt chơi của nhân vật.
- Cả hai đều kiểm tra xem nhân vật đã chết chưa (!character.IsDead.Value) trước khi xử lý để tránh gọi hàm chết nhiều lần.

Khác nhau:

Đặc điểm	Spike (Gai)	Water (Nước/Hố)
Cơ chế kích hoạt	Dựa vào va chạm vật lý giữa 2 Collider (OnCollisionStay2D).	Dựa vào tọa độ vị trí (thường kiểm tra transform.position.y < 0f trong Update).
Điều kiện hướng	Phải chạm vào đầu nhọn của gai (kiểm tra isTop qua normal.y < -0.7f).	Chỉ cần rơi xuống quá cao độ cho phép là chết, không xét hướng.
Hiệu ứng sau chết	Sử dụng FixedJoint2D để dính xác nhân vật vào gai.	Nhân vật thường biến mất hoặc rơi tự do khỏi màn hình.
Tham số máu	Gọi Die(true) (có phun máu).	Thường gọi Die() mặc định hoặc xử lý riêng cho hiệu ứng nước.

Thay đổi 1: Tốc độ chạy

Thay đổi 2: Lực nhảy

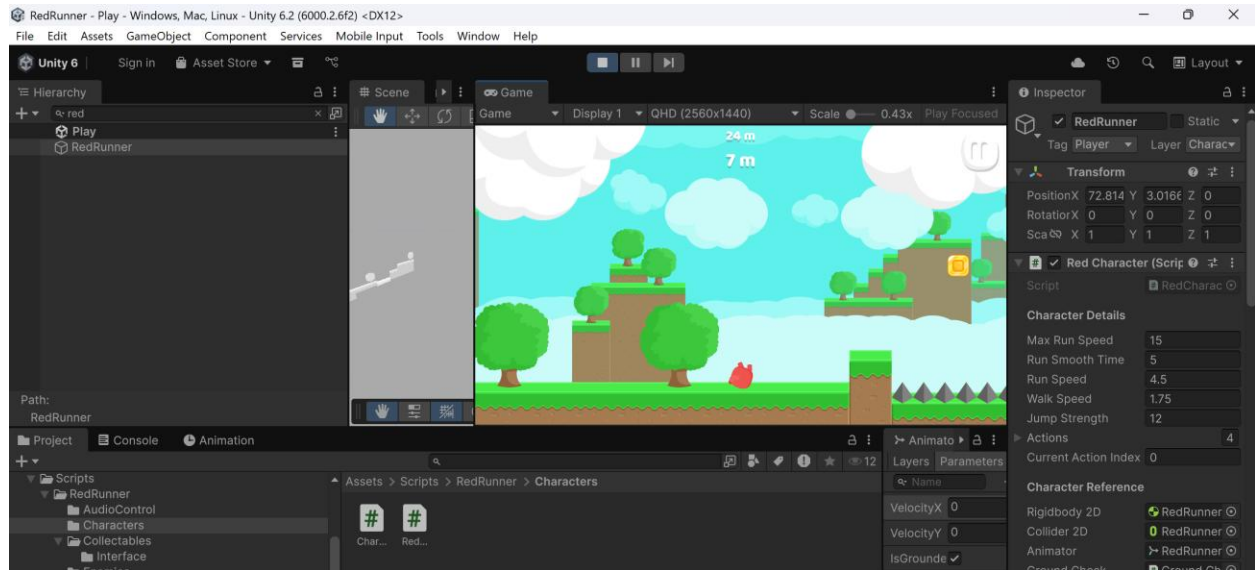
1. Chọn nhân vật RedRunner trong Hierarchy
2. Trong Inspector, tìm component Red Character (script)

3. Tìm thuộc tính Max Run Speed (giá trị mặc định: 8)

4. Thay đổi thành 15

5. Nhấn Play → quan sát sự khác biệt

6. Ghi nhận: Giá trị gốc: 9 → Giá trị mới: 15 → Kết quả: tốc độ max nhanh hơn



Thay đổi 2: Lực nhảy

Ghi nhận: Giá trị gốc: 12 → Giá trị mới: 24 → Kết quả: nhảy cao hơn

Thay đổi 3: Trọng lực

Thay đổi 4: Thêm Coin vào Scene

Task 3.2: Chụp ảnh minh chứng (10 phút)

Chụp ít nhất 6 screenshots sau:

✅ Checkpoint 3: Có ít nhất 6 screenshots minh chứng, ghi nhận đầy đủ các thay đổi

PHẦN 4: Git, GitHub & Nộp bài (30 phút)

Task 4.1: Commit và Push thay đổi (10 phút)

Mở Terminal (hoặc Git Bash) trong thư mục dự án:

2. Ghi nhận giá trị gốc

3. Tăng lên gấp đôi

4. Nhấn Play → quan sát

5. Ghi nhận: Giá trị gốc: → Giá trị mới: → Kết quả: ____

1. Chọn nhân vật RedRunner

2. Tìm component Rigidbody 2D → thuộc tính Gravity Scale

3. Thay đổi giá trị (thử: 0.5 và 3.0)

4. Nhấn Play → quan sát với từng giá trị

5. Ghi nhận: Giá trị gốc: → Giá trị 0.5: → Giá trị 3.0: ____

1. Mở tab Project → navigate đến Assets/Prefabs/Collectables/

2. Kéo thả prefab Coin vào Scene View

3. Sử dụng Move Tool (W) để đặt Coin ở vị trí dễ nhìn thấy

4. Nhấn Play → chạy đến vị trí Coin → thu thập

5. Ghi nhận: Coin có hoạt động đúng không? Có hiệu ứng gì xảy ra?